

Lab 2 Report – LLMs for dialogue systems

Introduction

In this lab, I implemented a voice chatbot using XState, SpeechState, and Ollama. The system allows the user to speak with a language model by speech recognition. The chatbot keeps conversation history and gives answers using the Llama 3.2 model via Ollama.

Implementation

The dialogue manager was done using an XState machine. The dialogue flow consists:

1. Speaking – the chatbot speaks.
2. Ask – the system listens to the user's speech.
3. ChatCompletion – the user's input is sent to Ollama, which generates a response.

Speech recognition was handled via SpeechState and Azure services. The chatbot uses a conversation history stored in a `messages[]` array. Each user utterance and assistant response is added to this history, and let the language model to keep context.

User and assistant messages are sent to the Ollama API, the response generated by the Llama 3.2 model. Error handling was implemented too. If the connection to Ollama fails, the system returns an error message.

Strengths

- * Conversation history is preserved using the `messages[]`, useful for multi-turn conversations.
- * Error handling improves strength when the language model is unavailable.
- * The state machine structure makes the dialogue flow easy to extend.

Weaknesses

- * The conversation history grows continuously, which may make long conversations.
- * The system depends on Ollama being active.
- * The dialogue manager follows a simple conversation loop and does not support more advanced dialogue.

Conclusion

The implemented system combines speech technology with a large language model to create an alive chatbot. The use of Ollama enables context-aware responses through history. The system can show how speech and language models can work together in a chatbot.